

Tema: Gestión de hilos

Presentado por: Jhon Andrioly Zenteno Sosa

2024-119066

1. **Al comparar el rendimiento de un programa multihilo con un monohilo, ¿qué diferencias observaste en el tiempo de ejecución? ¿Cómo influye la gestión adecuada de hilos en el rendimiento general del programa?**
 - El tiempo de CPU registrado con 1 hilo fue menor que el obtenido con 5 y 10 hilos, esto se debe a que la tarea que realiza cada hilo es sumamente pequeña.
 - La gestión de hilos mejora el rendimiento cuando el problema a resolver es intensivo en cómputo o involucra operaciones de entrada/salida bloqueantes que pueden paralelizarse. Sin embargo, si se crean hilos sin una tarea que justifique el costo de su creación, o si se añade sincronización excesiva con bloqueos de mutex constantes, el rendimiento se degrada en lugar de mejorar.
2. **¿Qué funciones de la biblioteca pthread utilizaste en tu implementación? Explica brevemente la finalidad de al menos tres de estas funciones y su importancia en la gestión de hilos.**

Se utilizaron las siguientes funciones de POSIX Threads:

- pthread_create
- pthread_join
- pthread_exit
- pthread_mutex_init
- pthread_mutex_lock
- pthread_mutex_unlock
- pthread_mutex_destroy

Explicación de tres funciones clave:

1. **pthread_create(&hilo, attr, funcion, arg):**
 - Inicializa y lanza un nuevo flujo de ejecución concurrente en el proceso, siendo así, el punto de partida fundamental para cualquier programa multihilo, asociando una rutina y sus parámetros para que se ejecuten en paralelo con el hilo principal.
2. **pthread_join(hilo, retval):**
 - Suspende la ejecución del hilo invocador hasta que el hilo indicado termine su ejecución, esto evita que el hilo principal finalice y destruya el proceso antes de que los hilos secundarios terminen su labor, además de permitir la recolección de resultados y liberar recursos del sistema.

3. `pthread_mutex_lock(&mutex)` y `pthread_mutex_unlock(&mutex)`:

- Bloquean y liberan respectivamente un objeto de exclusión mutua, para así delimitar la sección crítica de un programa garantizando que solo un hilo a la vez acceda a un recurso compartido, impidiendo condiciones de carrera e inconsistencias de datos.

3. ¿Cómo se comportó el programa al aumentar el número de hilos más allá de un cierto punto? ¿Hubo un límite en el que el rendimiento no mejoró o incluso disminuyó? Explica por qué esto puede ocurrir en un entorno multihilo.

- Al incrementar de 1 a 5 y luego a 10 hilos, el tiempo total medido aumentó.
- Sí, el rendimiento no mejoró en ningún momento al añadir más hilos.
- El rendimiento no mejoró debido a tres factores:
 - **Sobrecarga del sistema:** Crear más hilos obliga al kernel a gastar tiempo y memoria reservando pilas, gestionando estructuras y haciendo cambios de contexto constantes.
 - **Serialización por Mutex:** El bloqueo (`pthread_mutex_lock/unlock`) forzó a los hilos a ejecutarse uno por uno en lugar de en paralelo, sumando esperas por el cerrojo.
 - **Contención de CPU:** Tener más hilos que núcleos físicos satura la cola del planificador, aumentando los tiempos de espera del procesador.

4. Durante la ejecución, ¿observaste alguna condición de carrera o comportamiento inesperado? Si es así, ¿cómo impactó esto en los resultados obtenidos y qué medidas se podrían tomar para prevenirlo?

- No hubo pérdida de datos ni fallo crítico, solo variación en el orden de salida de los mensajes por consola
- Si los hilos hubiesen modificado variables compartidas, como un contador global sin protección, se habrían sobrescrito datos y perdido cálculos por accesos simultáneos a la memoria.
- Medidas preventivas:
 - Usar exclusión mutua con cerrojos (`pthread_mutex_lock` y `pthread_mutex_unlock`).
 - Emplear operaciones atómicas de hardware para contadores o valores numéricos simples.
 - Implementar semáforos o variables de condición si se necesita un orden de turno estricto.